1.Design and implement a program to simulate the translation of a logical address into a physical address using base and limit registers. The program should accept logical address as input, check for validity, and compute the corresponding physical address.Display appropriate messages for valid and invalid addresses.

CODE:

```
#include <stdio.h>

int main()
{
    // Variable declaration
    int base, limit;
    int logical, physical;

    // Get the base register value
    printf("Enter Base Register: ");
    scanf("%d", &base);

    // Get the limit register value
    printf("Enter Limit Register: ");
    scanf("%d", &limit);

    // Get the logical address
    printf("Enter Logical Address: ");
    scanf("%d", &logical);

    // Check whether the logical address is valid
    if (logical >= 0 && logical < limit)
    {
        // Calculate physical address
        physical = base + logical;

        printf("\nLogical Address is Valid\n");
        printf("Physical Address = %d\n", physical);
    }
    else
    {
        // Address is outside the limit
        printf("\nInvalid Logical Address\n");
    }

    return 0;
}
```

OUTPUT:

```
Enter Base Register: 1000
Enter Limit Register: 500
Enter Logical Address: 250

Logical Address is Valid
```

Physical Address = 1250

2.Write a program to demonstrate the paging memory management scheme. The program should divide memory into fixed-size pages and frames, accept page number and offset as input, and compute the corresponding physical address using a page table.

CODE:

```
#include <stdio.h>

int main()
{
    int pageTable[10], n, page, offset;
    int frame, physical;

    // Input page table
    printf("Enter number of pages: ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++)
    {
        printf("Frame for Page %d: ", i);
        scanf("%d", &pageTable[i]);
    }

    // Input logical address
    printf("Enter Page Number: ");
    scanf("%d", &page);

    printf("Enter Offset: ");
    scanf("%d", &offset);

    if(page < n)
    {
        frame = pageTable[page];
        physical = frame * 100 + offset;

        printf("Frame Number = %d\n", frame);
        printf("Physical Address = %d\n", physical);
    }
    else
    {
        printf("Invalid Page Number\n");
    }

    return 0;
}
```

OUTPUt:

```
Enter number of pages: 3
```

```
Frame for Page 0: 2
Frame for Page 1: 5
Frame for Page 2: 7
Enter Page Number: 1
Enter Offset: 25

Frame Number = 5
Physical Address = 525
```

3.Develop a program to simulate a page table. The program should allow the user to input page table entries and logical addresses, then map them to physical addresses. Clearly show how page numbers are translated into frame numbers.

CODE:

```
#include <stdio.h>

int main()
{
    int page[10], n, p, offset;

    // Enter number of pages
    printf("Enter number of pages: ");
    scanf("%d", &n);

    // Store frame number for each page
    for(int i = 0; i < n; i++)
    {
        printf("Frame for Page %d: ", i);
        scanf("%d", &page[i]);
    }

    // Enter logical address
    printf("Enter Page Number: ");
    scanf("%d", &p);

    printf("Enter Offset: ");
    scanf("%d", &offset);

    // Translate page number into frame number
    if(p >= 0 && p < n)
    {
        printf("\nPage %d -> Frame %d\n", p, page[p]);
        printf("Physical Address = %d\n", page[p] * 100 +
offset);
    }
    else
    {
        printf("\nInvalid Page Number");
    }
```

```
    return 0;
}
```

OUTPUT:

```
Enter number of pages: 3
Frame for Page 0: 4
Frame for Page 1: 1
Frame for Page 2: 6
Enter Page Number: 2
Enter Offset: 20

Page 2 -> Frame 6
Physical Address = 620
```

4.Implement a program to simulate the working of a Translation Lookaside Buffer (TLB). The program should demonstrate how TLB improves memory access time by calculating the effective memory access time with and without TLB.

CODE:

```
#include <stdio.h>

int main()
{
    float memoryTime, tlbTime, hitRatio;
    float withTLB, withoutTLB;

    // Get input values
    printf("Enter Memory Access Time: ");
    scanf("%f", &memoryTime);

    printf("Enter TLB Access Time: ");
    scanf("%f", &tlbTime);

    printf("Enter TLB Hit Ratio (0 to 1): ");
    scanf("%f", &hitRatio);

    // Calculate access time
    withoutTLB = 2 * memoryTime;
    withTLB = hitRatio * (tlbTime + memoryTime) +
              (1 - hitRatio) * (tlbTime + 2 * memoryTime);

    // Display result
    printf("\nWithout TLB = %.2f ns", withoutTLB);
    printf("\nWith TLB = %.2f ns", withTLB);

    return 0;
}
```

OUTPUT:

```
Enter Memory Access Time: 100
Enter TLB Access Time: 20
Enter TLB Hit Ratio (0 to 1): 0.8

Without TLB = 200.00 ns
With TLB = 140.00 ns
```

5.Write a program to illustrate memory protection using base and limit registers. The program should verify whether a given logical address lies within the valid range and prevent unauthorized access.

CODE:

```
#include <stdio.h>

int main()
{
    int base, limit, logical, physical;

    // Get register values
    printf("Enter Base Register: ");
    scanf("%d", &base);

    printf("Enter Limit Register: ");
    scanf("%d", &limit);

    // Enter logical address
    printf("Enter Logical Address: ");
    scanf("%d", &logical);

    // Check memory protection
    if(logical >= 0 && logical < limit)
    {
        physical = base + logical;
        printf("\nAccess Granted\n");
        printf("Physical Address = %d\n", physical);
    }
    else
    {
        printf("\nAccess Denied\n");
        printf("Invalid Logical Address\n");
    }

    return 0;
}
```

OUTPUT:

```
Enter Base Register: 1000
Enter Limit Register: 500
```

```
Enter Logical Address: 250

Access Granted
Physical Address = 1250
```